

# Classification via Local Search

## Algorithm Design

Jon Kleinberg, Éva Tardos

CS 210 Report  
J Franco Ray

May 26, 2016

# Outline

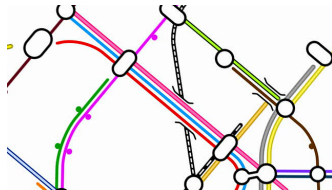
## 1 Network Flow

- Definition
- Maximum Flow Problem
- Image Segmentation

## 2 Local Search

- State-Flipping Algorithm
- Classification via Local Search
- Analyzing the Algorithm

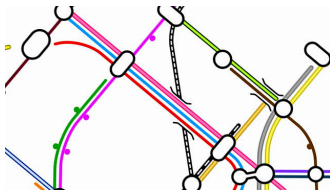
# Network Flow



*Motivation:* Transportation networks carry some sort of traffic whose nodes act as 'switches' passing traffic between edges.

- capacities on the edges (how much they can carry?)
- source nodes (of traffic)
- sink nodes (destination), absorbs traffic
- the traffic itself

# Network Flow

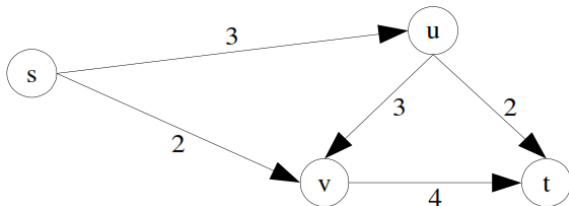


*Motivation:* Transportation networks carry some sort of traffic whose nodes act as 'switches' passing traffic between edges.

- capacities on the edges (how much they can carry?)
- source nodes (of traffic)
- sink nodes (destination), absorbs traffic
- the traffic itself

*Question:* How much traffic can a network handle?

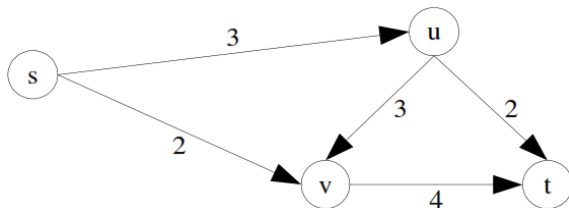
# Network Flow



A flow network is a directed graph  $G = (V, E)$ :

- $c_e$  - capacity of the edge, nonnegative
- $s$  - single source node
- $t$  - single sink node
- internal nodes (traffic junctions)

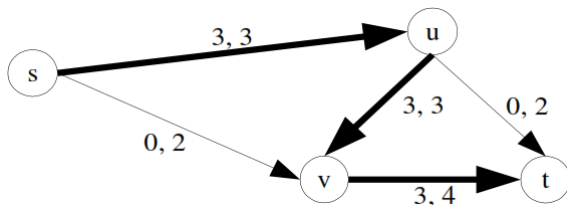
# Network Flow



Some assumptions:

- no edge enters  $s$  and no edge leaves  $t$
- at least one edge incident to each node
- all capacities are integers

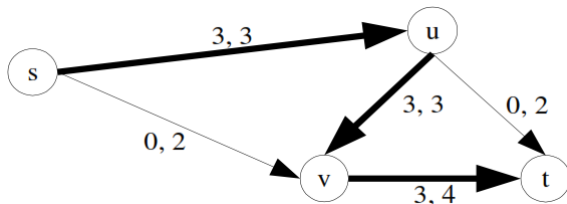
# Defining flow



A network flow is a function  $f : E \rightarrow \mathbb{R}^+$  that represents the amount of flow on an edge.

- For each  $e \in E$ ,  $0 \leq f(e) \leq c_e$

# Defining flow



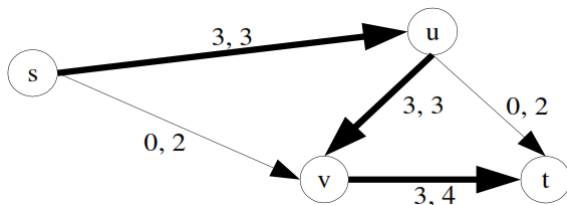
A network flow is a function  $f : E \rightarrow \mathbb{R}^+$  that represents the amount of flow on an edge.

- For each  $e \in E$ ,  $0 \leq f(e) \leq c_e$
- For each node  $v$  other than  $s, t$ ,

$$\sum_{e \text{ into } v} f(e) = \sum_{e \text{ out of } v} f(e)$$



# Defining flow



A network flow is a function  $f : E \rightarrow \mathbb{R}^+$  that represents the amount of flow on an edge.

- For each  $e \in E$ ,  $0 \leq f(e) \leq c_e$
- For each node  $v$  other than  $s, t$ ,

$$\sum_{e \text{ into } v} f(e) = \sum_{e \text{ out of } v} f(e)$$

- $v(f)$  is the amount of flow generated at the source

# Maximum Flow Problem

## Maximum Flow Problem

Given a flow network, find a flow of maximum possible value.

Suppose we divide the nodes of the graph into two sets,  $A$  and  $B$ , so that  $s \in A$  and  $t \in B$ .

# Maximum Flow Problem

## Maximum Flow Problem

Given a flow network, find a flow of maximum possible value.

Suppose we divide the nodes of the graph into two sets,  $A$  and  $B$ , so that  $s \in A$  and  $t \in B$ .

- Any flow from  $s$  to  $t$  must cross from  $A$  to  $B$  at some point and use up some of the edges from  $A$  to  $B$ .

# Maximum Flow Problem

## Maximum Flow Problem

Given a flow network, find a flow of maximum possible value.

Suppose we divide the nodes of the graph into two sets,  $A$  and  $B$ , so that  $s \in A$  and  $t \in B$ .

- Any flow from  $s$  to  $t$  must cross from  $A$  to  $B$  at some point and use up some of the edges from  $A$  to  $B$ .
- The capacity of the cut:  $c(A, B) = \sum_{e \text{ out of } A} c_e$ .

# Maximum Flow Problem

## Maximum Flow Problem

Given a flow network, find a flow of maximum possible value.

Suppose we divide the nodes of the graph into two sets,  $A$  and  $B$ , so that  $s \in A$  and  $t \in B$ .

- Any flow from  $s$  to  $t$  must cross from  $A$  to  $B$  at some point and use up some of the edges from  $A$  to  $B$ .
- The capacity of the cut:  $c(A, B) = \sum_{e \text{ out of } A} c_e$ .
- $v(f) \leq c(A, B)$

# Maximum Flow Problem

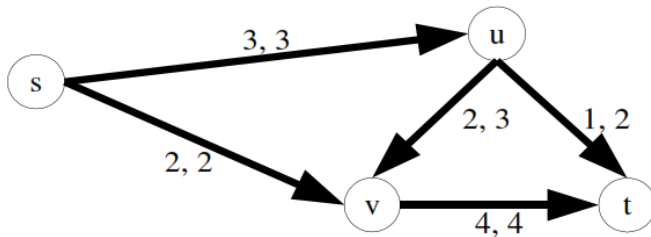
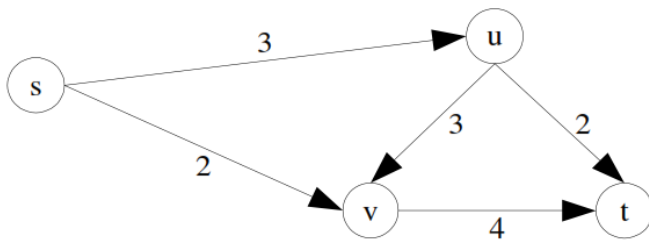
## Maximum Flow Problem

Given a flow network, find a flow of maximum possible value.

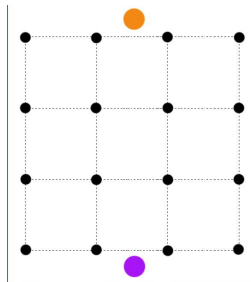
Suppose we divide the nodes of the graph into two sets,  $A$  and  $B$ , so that  $s \in A$  and  $t \in B$ .

- Any flow from  $s$  to  $t$  must cross from  $A$  to  $B$  at some point and use up some of the edges from  $A$  to  $B$ .
- The capacity of the cut:  $c(A, B) = \sum_{e \text{ out of } A} c_e$ .
- $v(f) \leq c(A, B)$
- The optimal cut(s) can be solved using *Ford-Fulkerson* algorithm: max flow equals min cut

# Example



# Image Segmentation

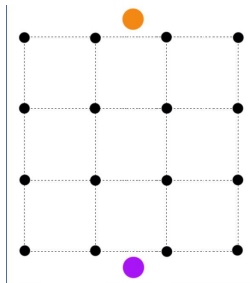


We wish to label each pixel in an image as FG/BG pixel.

- $a_i, b_i \geq 0$ : likelihood that  $i$  belongs to FG/BG
- $p_{ij} \geq 0$ : separation penalty for classifying  $i$  and  $j$  differently



# Image Segmentation



## Segmentation Problem

Find a partition of the set of pixels to maximize

$$q(A, B) = \sum_{i \in A} a_i + \sum_{j \in B} b_j - \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}| = 1}} p_{ij}$$

# Reduction to Max Flow Problem

- Let  $Q = \sum_i (a_i + b_i)$ . Then

$$\sum_{i \in A} a_i + \sum_{j \in B} b_j = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j$$

# Reduction to Max Flow Problem

- Let  $Q = \sum_i (a_i + b_i)$ . Then

$$\sum_{i \in A} a_i + \sum_{j \in B} b_j = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j$$

- So that

$$q(A, B) = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j - \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}|=1}} p_{ij}$$

# Reduction to Max Flow Problem

- Let  $Q = \sum_i (a_i + b_i)$ . Then

$$\sum_{i \in A} a_i + \sum_{j \in B} b_j = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j$$

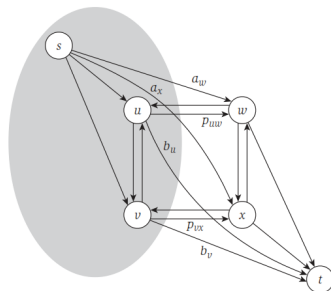
- So that

$$q(A, B) = Q - \sum_{i \in A} b_i - \sum_{j \in B} a_j - \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}|=1}} p_{ij}$$

- We see that maximizing  $q(A, B)$  is the same as minimizing

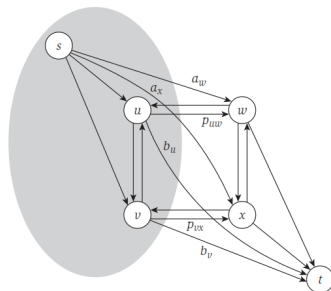
$$q'(A, B) = \sum_{i \in A} b_i + \sum_{j \in B} a_j + \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}|=1}} p_{ij}$$

# Reduction to Max Flow Problem



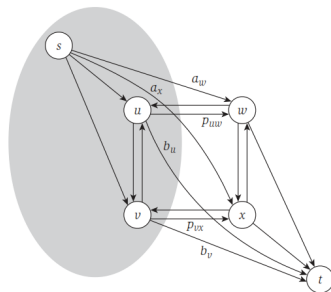
- Create source  $s$  and sink  $t$  to represent FG and BG, resp

# Reduction to Max Flow Problem



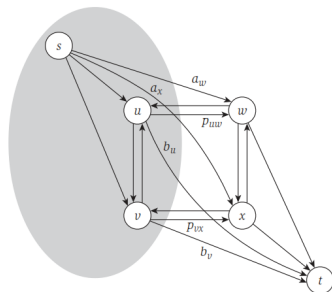
- Create source  $s$  and sink  $t$  to represent FG and BG, resp
- Attach  $s$  and  $t$  to every pixel and use  $a_i$  and  $b_i$  to define edge capacities on  $s$ ,  $t$ , and each  $i$ .

# Reduction to Max Flow Problem



- Create source  $s$  and sink  $t$  to represent FG and BG, resp
- Attach  $s$  and  $t$  to every pixel and use  $a_i$  and  $b_i$  to define edge capacities on  $s$ ,  $t$ , and each  $i$ .
- Replace undirected edges between each pixel to two directed edges.
- Produces flow network  $G' = (V', E')$

# Reduction to Max Flow Problem

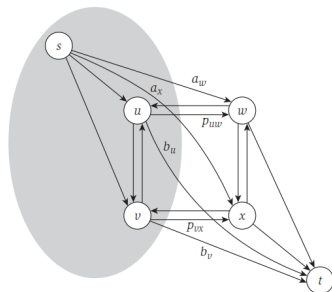


Group the edges that cross the cut into three

- Edges  $(s, j), j \in B$ , contributes  $a_j$  to  $c(A, B)$



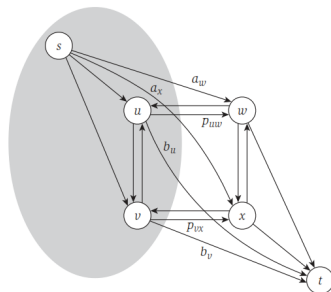
# Reduction to Max Flow Problem



Group the edges that cross the cut into three

- Edges  $(s, j), j \in B$ , contributes  $a_j$  to  $c(A, B)$
- Edges  $(i, t), i \in A$ , contributes  $b_i$  to  $c(A, B)$

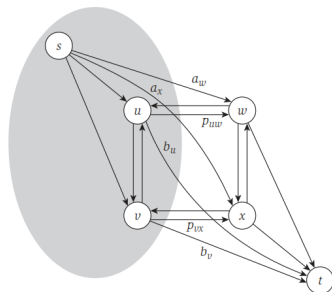
# Reduction to Max Flow Problem



Group the edges that cross the cut into three

- Edges  $(s, j), j \in B$ , contributes  $a_j$  to  $c(A, B)$
- Edges  $(i, t), i \in A$ , contributes  $b_i$  to  $c(A, B)$
- Edges  $(i, j), i \in A, j \in B$ , contributes  $p_{ij}$  to  $c(A, B)$

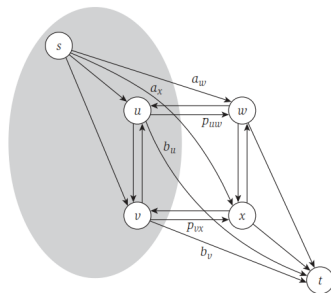
# Reduction to Max Flow Problem



Group the edges that cross the cut into three

- Edges  $(s, j), j \in B$ , contributes  $a_j$  to  $c(A, B)$
- Edges  $(i, t), i \in A$ , contributes  $b_i$  to  $c(A, B)$
- Edges  $(i, j), i \in A, j \in B$ , contributes  $p_{ij}$  to  $c(A, B)$
- Adding them all we get  $c(A, B) = q'(A, B)$

# Reduction to Max Flow Problem



$$c(A, B) = \sum_{i \in A} b_i + \sum_{j \in B} a_j + \sum_{\substack{(i,j) \in E \\ |A \cap \{i,j\}| = 1}} p_{ij} = q'(A, B)$$

Solution is obtained by a minimum-cut algorithm in the graph  $G'$ . The partition obtained by deleting  $s$  and  $t$  maximizes the reward  $q(A, B)$ .

# State-Flipping Algorithm

Used to obtain neighbors for local searches in graph partitions.

## State-Flipping Algorithm

If there exists a node  $u$  s.t.

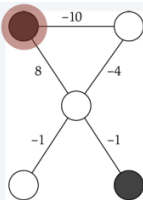
$$\sum_{i \in \text{edges from } u \text{ into } A} w(i) > \sum_{j \in \text{edges from } u \text{ out of } A} w(j)$$

then  $u$  should be moved on the other partition.

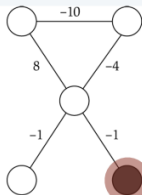
Two labelings are neighboring solutions if one can be obtained from the other by executing a single node flip.

# State-Flipping Algorithm

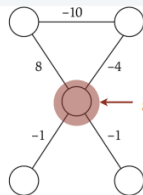
unsatisfied node  
 $10 - 8 > 0$



(a)

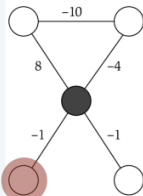


(b)

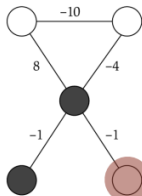


(c)

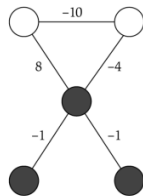
unsatisfied node  
 $8 - 4 - 1 - 1 > 0$



(d)



(e)



(f)

stable

# Choosing a Neighbor Relation

## 2-approximation

Let  $(A, B)$  be a partition that is a local optimum obtained from the single-flip nbhd. Let  $(A^*, B^*)$  be a globally optimal partition. Then

$$2 \cdot w(A, B) \geq w(A^*, B^*).$$

# Choosing a Neighbor Relation

## 2-approximation

Let  $(A, B)$  be a partition that is a local optimum obtained from the single-flip nbhd. Let  $(A^*, B^*)$  be a globally optimal partition. Then

$$2 \cdot w(A, B) \geq w(A^*, B^*).$$

## $k$ -flip nbhd, $k > 1$

Two labelings are neighbors under the  $k$ -flip rule if one can be obtained from the other by flipping at most  $k$  nodes.



# Classification via Local Search

- To obtain good performance, we use an exponentially large nbhd
- But we will need more sophisticated algo to find improving neighbor if it exists.

# Classification via Local Search

- To obtain good performance, we use an exponentially large nbhd
- But we will need more sophisticated algo to find improving neighbor if it exists.
- Recall the basic ISP. We now consider labeling with more than two labels.

# General Labeling Problem

Given a graph  $G = (V, E)$  and a set of labels  $L$ . The goal is to label each node to minimize penalty.

- For each edge  $(i, j) \in E$ , we have a separation penalty  $p_{ij}$  for labeling  $i$  and  $j$  differently.
- For each node  $i \in V$  and each label  $a \in L$ , we have a penalty  $c(i, a) \geq 0$  for assigning label  $a$  to node  $i$ .

# General Labeling Problem

Given a graph  $G = (V, E)$  and a set of labels  $L$ . The goal is to label each node to minimize penalty.

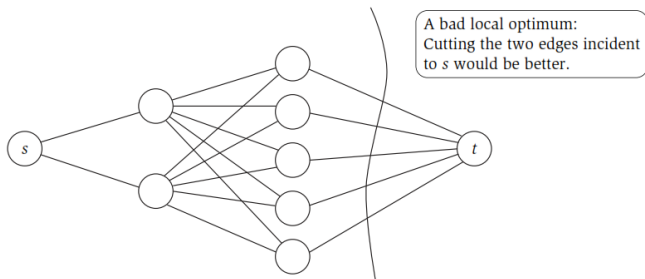
- For each edge  $(i, j) \in E$ , we have a separation penalty  $p_{ij}$  for labeling  $i$  and  $j$  differently.
- For each node  $i \in V$  and each label  $a \in L$ , we have a penalty  $c(i, a) \geq 0$  for assigning label  $a$  to node  $i$ .

## Labeling Problem

Find a labeling  $f : V \rightarrow L$  that minimizes the total penalty

$$\Phi(f) = \sum_{i \in V} c(i, f(i)) + \sum_{\substack{(i,j) \in E \\ f(i) \neq f(j)}} p_{ij}$$

# First Attempt: the Single-Flip Rule



**Figure 12.5** An instance of the Minimum  $s$ - $t$  Cut Problem, where all edges have capacity 1.

- Two labelings are neighbors if one can be obtained from the other by flipping a single node.
- Can lead to quite poor local optima, not a good approx algo for min cut problem

# Considering two Labels at a Time

- We pick two labels  $a$  and  $b$  in labeling  $f$ . In a single local step, allow any subset of nodes to flip labels  $a \leftrightarrow b$ .

# Considering two Labels at a Time

- We pick two labels  $a$  and  $b$  in labeling  $f$ . In a single local step, allow any subset of nodes to flip labels  $a \leftrightarrow b$ .
- Two labelings  $f$  and  $f'$  are neighbors if there are two labels  $a, b \in L$  such that  $\forall c \in L \neq a, b, \forall i \in V$  we have

$$f(i) = c \Leftrightarrow f'(i) = c$$

# Considering two Labels at a Time

- We pick two labels  $a$  and  $b$  in labeling  $f$ . In a single local step, allow any subset of nodes to flip labels  $a \leftrightarrow b$ .
- Two labelings  $f$  and  $f'$  are neighbors if there are two labels  $a, b \in L$  such that  $\forall c \in L \neq a, b, \forall i \in V$  we have

$$f(i) = c \Leftrightarrow f'(i) = c$$

- Note that  $f$  can have exponentially many neighbors,  $\leq 2^x$ . However,



# Considering two Labels at a Time

- We pick two labels  $a$  and  $b$  in labeling  $f$ . In a single local step, allow any subset of nodes to flip labels  $a \leftrightarrow b$ .
- Two labelings  $f$  and  $f'$  are neighbors if there are two labels  $a, b \in L$  such that  $\forall c \in L \neq a, b, \forall i \in V$  we have

$$f(i) = c \Leftrightarrow f'(i) = c$$

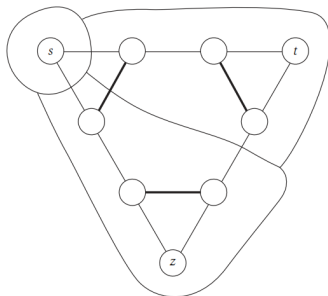
- Note that  $f$  can have exponentially many neighbors,  $\leq 2^x$ . However,

## Theorem 12.8

If a labeling  $f$  is not locally optimal for the nbhd, a better neighbor can be found via  $k^2$  min cut computations. *There are fewer than  $k^2$  pairs of distinct labels,  $kC2$ , and we try each pair separately. To find best improved labeling via swapping two labels is exactly SP.*

# Considering two Labels at a Time

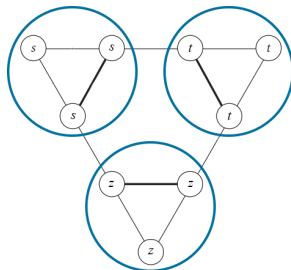
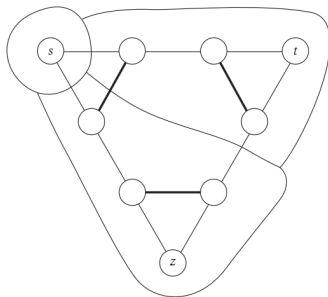
Solve two labels optimally but local optima can still be bad



- Three nodes required to keep their labels
- Other nodes should get one of the labels on the same side of triangle
- Give each node large penalty for labels not allowed
- Light edges have penalty 1, heavy edges have penalty  $\infty$

# Considering two Labels at a Time

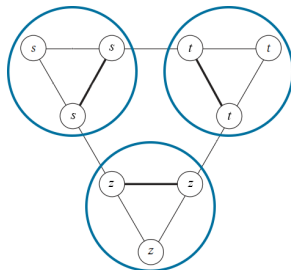
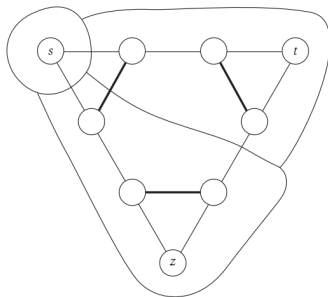
Solve two labels optimally but local optima can still be bad



- Light edges have penalty 1, heavy edges have penalty  $\infty$
- Current labeling has penalty  $M + 3$  but locally optimal.

# Considering two Labels at a Time

Solve two labels optimally but local optima can still be bad



- Light edges have penalty 1, heavy edges have penalty  $\infty$
- Current labeling has penalty  $M + 3$  but locally optimal.
- Global optimal is only 3

# A More Local Search Nbhd that Works

*Idea:* Relabel nodes of different labels in a single step

- Pick one label  $a \in L$  and focus on nodes not labeled as  $a$
- As a single local step, allow subset of these nodes to change their labels to  $a$ .

# A More Local Search Nbhd that Works

*Idea:* Relabel nodes of different labels in a single step

- Pick one label  $a \in L$  and focus on nodes not labeled as  $a$
- As a single local step, allow subset of these nodes to change their labels to  $a$ .
- Two labelings  $f$  and  $f'$  are neighbors if there is a label  $a \in L$  s.t.  
 $\forall i \in V, f'(i) = f(i) \vee f'(i) = a$ .
- We can find the best neighbor of  $f$  via  $k$  min cut computations and a local optimum is a 2-approximation for the labeling problem.

# Finding an good neighbor

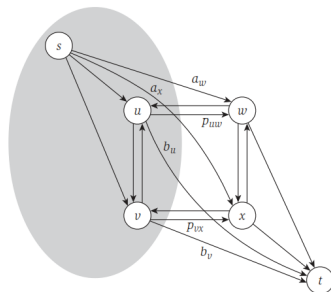
Consider a label  $a$ .

## Claim

The best relabeling in which nodes may change their labels to  $a$  can be found min cut computation.

The construction of the network flow  $G' = (V', E')$  is analogous to the min cut computation done for two-label ISP.

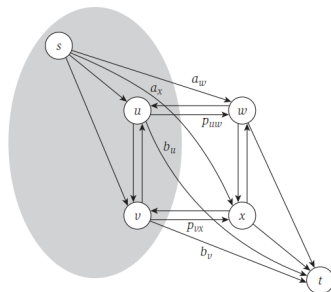
# Finding a good neighbor



- Introduce a source  $s$  that represent label  $a$  and sink  $t$  to represent the original labels on the nodes.

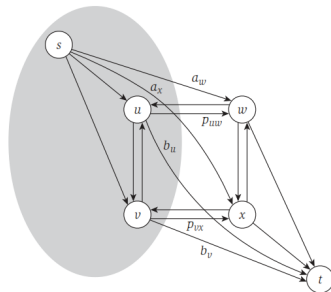


# Finding a good neighbor



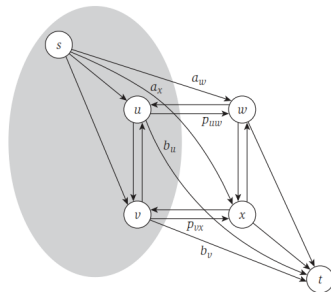
- Introduce a source  $s$  that represent label  $a$  and sink  $t$  to represent the original labels on the nodes.
- Find the min cut in  $G'$  and relabel all nodes on the  $s$ -side to label  $a$  while the nodes on the  $t$ -side will keep their orig labels.

# Finding a good neighbor



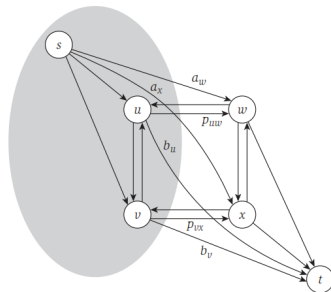
- Add  $G$  to  $G'$  as well as edges  $(s, i)$  and  $(i, t)$  to  $E'$ .

# Finding a good neighbor



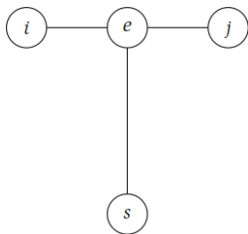
- Add  $G$  to  $G'$  as well as edges  $(s, i)$  and  $(i, t)$  to  $E'$ .
- Edge  $(i, t)$  will have capacity  $c(i, a)$ , as cutting them places node  $i$  on the source side and labeling it with  $a$ .

# Finding a good neighbor



- Add  $G$  to  $G'$  as well as edges  $(s, i)$  and  $(i, t)$  to  $E'$ .
- Edge  $(i, t)$  will have capacity  $c(i, a)$ , as cutting them places node  $i$  on the source side and labeling it with  $a$ .
- Edge  $(s, i)$  will have capacity  $c(i, f(i))$  if  $f(i) \neq a$ , and  $\infty$  if  $f(i) = a$  as cutting them places node  $i$  on the sink side and retaining its current label.

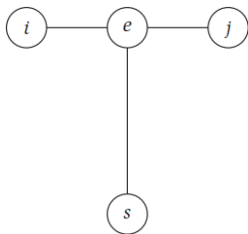
# Finding a good neighbor



If both  $i$  and  $j$  are on the same side, then the edge connecting them is not cut, yet  $i$  and  $j$  are separated if  $f(i) \neq f(j)$ .

- For an edge  $(i, j)$ , if  $f(i) = f(j)$  or one if  $i$  or  $j$  is labeled  $a$ , then we add an edge  $(i, j)$  to  $E'$  with capacity  $p_{ij}$ .

# Finding a good neighbor



If both  $i$  and  $j$  are on the same side, then the edge connecting them is not cut, yet  $i$  and  $j$  are separated if  $f(i) \neq f(j)$ .

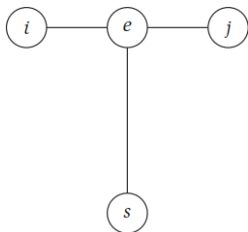
- For an edge  $(i, j)$ , if  $f(i) = f(j)$  or one if  $i$  or  $j$  is labeled  $a$ , then we add an edge  $(i, j)$  to  $E'$  with capacity  $p_{ij}$ .
- For the rest of the edges, add an extra node  $e$  to  $V'$  corresponding to edge  $e$ , and all the edges  $(i, e)$ ,  $(e, j)$ ,  $(e, s)$  with capacity  $p_{ij}$ .

# Finding a good neighbor

## Theorem 12.9

Given a labeling  $f$  and a label  $a$ , the min cut in the graph  $G' = (V', E')$  corresponds to the minimum penalty neighbor of labeling  $f$  obtained by relabeling a subset of nodes to label  $a$ . As a result, a better neighbor can be found via  $k$  min cut computations, one for each label in  $L$ .

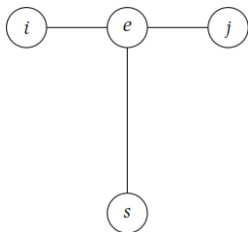
# Finding a good neighbor



- The large value of  $\infty$  ensures that it will not cut heavy edges.

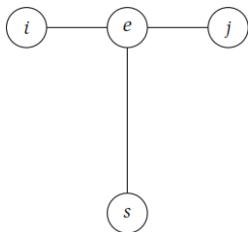


# Finding a good neighbor



- The large value of  $\infty$  ensures that it will not cut heavy edges.
- Consider a node  $e$  in  $G'$  corresponding to an edge  $e = (i, j) \in E$ . It has three adjacent edges each with capacity  $p_{ij}$ .

# Finding a good neighbor



- The large value of  $\infty$  ensures that it will not cut heavy edges.
- Consider a node  $e$  in  $G'$  corresponding to an edge  $e = (i, j) \in E$ . It has three adjacent edges each with capacity  $p_{ij}$ .
- Given any partition, we can place  $e$  so that at most one of these edges is cut.

# Finding a good neighbor

We'll call a cut *good* if

- no heavy edge is cut
- for all nodes corresponding to edges in  $E$ , at most one of the adjacent edges is cut.

Good cuts in  $G'$  are in *one-to-one* correspondence with relabelings of  $f$

# Finding a good neighbor

We'll call a cut *good* if

- no heavy edge is cut
- for all nodes corresponding to edges in  $E$ , at most one of the adjacent edges is cut.

Good cuts in  $G'$  are in *one-to-one* correspondence with relabelings of  $f$

- The edges  $(s, i)$  and  $(i, t)$  contribute exactly the assignment penalty to the capacity of the cut.
- The edges  $(i, j)$  contribute exactly the separation penalty

# Finding a good neighbor

We'll call a cut *good* if

- no heavy edge is cut
- for all nodes corresponding to edges in  $E$ , at most one of the adjacent edges is cut.

Good cuts in  $G'$  are in *one-to-one* correspondence with relabelings of  $f$

- The edges  $(s, i)$  and  $(i, t)$  contribute exactly the assignment penalty to the capacity of the cut.
- The edges  $(i, j)$  contribute exactly the separation penalty

## Finding a good neighbor

As a result, the capacity of a good cut  $c(A, B)$  is exactly the same as the penalty of the corresponding labeling  $\Phi(f)$ , *and so the min cut corresponds to the best relabeling of  $f$ .*

# Analyzing the Algorithm

We'll show that any local optimum under a new neighbor relation is a *2-approximation* to the minimum possible penalty.

- Consider an optimal labeling  $f^*$  and for a label  $a \in L$ , let  $V_a^* = \{i : f^*(i) = a\}$  be the set of nodes labeled by  $a$  in  $f^*$ .

# Analyzing the Algorithm

We'll show that any local optimum under a new neighbor relation is a *2-approximation* to the minimum possible penalty.

- Consider an optimal labeling  $f^*$  and for a label  $a \in L$ , let  $V_a^* = \{i : f^*(i) = a\}$  be the set of nodes labeled by  $a$  in  $f^*$ .
- Consider a locally optimal labeling  $f$ . We obtain a neighbor  $f_a$  by starting with  $f$  and relabeling all nodes in  $V_a^*$  to  $a$ .

# Analyzing the Algorithm

We'll show that any local optimum under a new neighbor relation is a *2-approximation* to the minimum possible penalty.

- Consider an optimal labeling  $f^*$  and for a label  $a \in L$ , let  $V_a^* = \{i : f^*(i) = a\}$  be the set of nodes labeled by  $a$  in  $f^*$ .
- Consider a locally optimal labeling  $f$ . We obtain a neighbor  $f_a$  by starting with  $f$  and relabeling all nodes in  $V_a^*$  to  $a$ .
- Hence,  $\Phi(f_a) \geq \Phi(f)$



# Analyzing the Algorithm

Now, consider the difference  $\Phi(f_a) - \Phi(f) \geq 0$ . *What quantities contribute to this difference?*

- The change in assignment penalties come from nodes  $i \in V_a^*$

# Analyzing the Algorithm

Now, consider the difference  $\Phi(f_a) - \Phi(f) \geq 0$ . *What quantities contribute to this difference?*

- The change in assignment penalties come from nodes  $i \in V_a^*$
- The separation penalties differ between the two labelings only in edges  $(i, j)$  that have at least one end in  $V_a^*$ .

# Analyzing the Algorithm

Now, consider the difference  $\Phi(f_a) - \Phi(f) \geq 0$ . *What quantities contribute to this difference?*

- The change in assignment penalties come from nodes  $i \in V_a^*$
- The separation penalties differ between the two labelings only in edges  $(i, j)$  that have at least one end in  $V_a^*$ .

## Theorem 12.10

For a labeling  $f$  and its neighbor  $f_a$ , we have

$$\begin{aligned}\Phi(f_a) - \Phi(f) \leq & \sum_{i \in V_a^*} [c(i, f^*(i)) - c(i, f(i))] \\ & + \sum_{(i,j) \text{ leaving } V_a^*} p_{ij} - \sum_{\substack{(i,j) \text{ in or leaving } V_a^* \\ f(i) \neq f(j)}} p_{ij}\end{aligned}$$

# Analyzing the Algorithm

## Theorem 12.11

For any locally optimal labeling  $f$  and any other labeling  $f^*$ , we have  $\Phi(f) \leq 2\Phi(f^*)$ . As a result, all local optima are good approximations to the global optimum.

*Proof.* Since  $0 \leq \Phi(f_a) - \Phi(f)$ , we have

$$0 \leq \sum_{a \in L} \left[ \sum_{i \in V_a^*} [c(i, f^*(i)) - c(i, f(i))] + \sum_{(i,j) \text{ leaving } V_a^*} p_{ij} - \sum_{\substack{(i,j) \text{ in or leaving } V_a^* \\ f(i) \neq f(j)}} p_{ij} \right]$$

$$\sum_{a \in L} \left[ \sum_{i \in V_a^*} c(i, f^*(i)) + \sum_{(i,j) \text{ leaving } V_a^*} p_{ij} \right] \geq \sum_{a \in L} \left[ \sum_{i \in V_a^*} c(i, f(i)) + \sum_{\substack{(i,j) \text{ in or leaving } V_a^* \\ f(i) \neq f(j)}} p_{ij} \right]$$

# Analyzing the Algorithm

- *left-hand side*: We get  $c(i, f^*(i))$  for all nodes  $i$ , which is exactly the assignment penalty of  $f^*$ . In addition, we get the term  $p_{ij}$  twice for each of the edges separated by  $f^*$ .

# Analyzing the Algorithm

- *left-hand side*: We get  $c(i, f^*(i))$  for all nodes  $i$ , which is exactly the assignment penalty of  $f^*$ . In addition, we get the term  $p_{ij}$  twice for each of the edges separated by  $f^*$ .
- *right-hand side*: We get  $c(i, f(i))$  for each node  $i$ , which is exactly the assignment penalty of  $f$ . In addition, we get the terms  $p_{ij}$  for edges separated by  $f$ . We get each such separation penalty at least once, and possibly twice if it is also separated by  $f^*$ .

# Analyzing the Algorithm

- *left-hand side*: We get  $c(i, f^*(i))$  for all nodes  $i$ , which is exactly the assignment penalty of  $f^*$ . In addition, we get the term  $p_{ij}$  twice for each of the edges separated by  $f^*$ .
- *right-hand side*: We get  $c(i, f(i))$  for each node  $i$ , which is exactly the assignment penalty of  $f$ . In addition, we get the terms  $p_{ij}$  for edges separated by  $f$ . We get each such separation penalty at least once, and possibly twice if it is also separated by  $f^*$ .
- In summary, we get the following.

$$\begin{aligned} 2\Phi(f^*) &\geq \sum_{a \in L} \left[ \sum_{i \in V_a^*} c(i, f^*(i)) + \sum_{(i,j) \text{ leaving } V_a^*} p_{ij} \right] \\ &\geq \sum_{a \in L} \left[ \sum_{i \in V_a^*} c(i, f(i)) + \sum_{\substack{(i,j) \text{ in or leaving } V_a^* \\ f(i) \neq f(j)}} p_{ij} \right] \geq \Phi(f) \end{aligned}$$

# Analyzing the Algorithm

*How fast does the algorithm find a local optimum?* We can only accept big improvements, recall MCP.

- Let  $\epsilon > 0$  be a constant.



# Analyzing the Algorithm

*How fast does the algorithm find a local optimum?* We can only accept big improvements, recall MCP.

- Let  $\epsilon > 0$  be a constant.
- For a given labeling  $f$ , we will consider a neighbor  $f'$  a *significant improvement* if  $\Phi(f') \leq (1 - \epsilon/3k)\Phi(f)$

# Analyzing the Algorithm

*How fast does the algorithm find a local optimum?* We can only accept big improvements, recall MCP.

- Let  $\epsilon > 0$  be a constant.
- For a given labeling  $f$ , we will consider a neighbor  $f'$  a *significant improvement* if  $\Phi(f') \leq (1 - \epsilon/3k)\Phi(f)$
- After at most  $\epsilon^{-1}k$  significant improvements, the penalty decreases by a constant factor; hence the algorithm will terminate in polynomial time.

# Analyzing the Algorithm

## Theorem 12.12

For any fixed  $\epsilon > 0$ , the version of the local search algorithm that only accepts significant improvements terminates in polynomial time and results in a labeling  $f$  such that  $\Phi(f) \leq (2 + \epsilon)\Phi(f^*)$  for any other labeling  $f^*$ .

*Proof.* Since  $\epsilon/3k \leq 1$  and  $\Phi(f) \leq 2\Phi(f^*)$  from Thm 12.11

$$\Phi(f) \leq (2 + \epsilon)\Phi(f^*)$$

adapting the proof from Thm 12.11.

Thank you for your attention!